



Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática

AED III - Trabalho Prático
Documento da Fase IV do TP

RELATÓRIO DE COMPRESSÃO DE DADOS: HUFFMAN vs. LZW

Este relatório explica como funcionam os métodos de compressão Huffman e LZW, analisando os resultados práticos obtidos no teste com um arquivo de 13.856 bytes.

1. Como Funcionam os Algoritmos?

- **Huffman (Foco em letras/bytes isolados):** Ele realiza uma contagem de quais bytes aparecem com mais frequência no arquivo. Os bytes mais frequentes ganham códigos bem curtos (com menos bits) e os mais raros ganham códigos mais longos. No momento da descompactação, ele usa uma tabela salva no começo do arquivo para traduzir esses códigos de volta aos bytes originais.
- **LZW (Foco em sequências de palavras/padrões):** Esse modelo não olha para letras isoladas, mas sim para sequências que se repetem. O LZW cria um "dicionário" automático enquanto lê o arquivo. Ao encontrar uma sequência de dados que já viu antes, ele a substitui por um único código numérico. Na descompactação, ele reconstrói esse mesmo dicionário sozinho, sem precisar que a tabela venha junto com o arquivo.

2. Resultados Práticos do Teste

O arquivo utilizado no teste é composto por estruturas binárias de tamanho fixo (structs), que possuem espaços preenchidos com zeros (padding).

1. Taxa de compressão — Huffman

- **Tamanho original:** 13.856 bytes
- **Tamanho comprimido:** 7.605 bytes
- **Cálculos:**

1. Taxa (comprimido/original) = $7605 / 13856 = 0,5489 = 54,89 \%$
2. Economia de espaço = $1 - 0,5489 = 45,11 \%$
3. Razão de compressão = $13856 / 7605 = 1,82 : 1$

O que isso significa? O Huffman reduziu o arquivo em aproximadamente 45%. O ganho vem da codificação por entropia: como os structs de tamanho fixo têm muitos bytes repetidos (padding 0x00 dos campos char[] e campos numéricos pequenos), esses bytes recebem códigos curtos. O ganho é moderado porque o Huffman explora apenas a frequência de bytes individuais, não sequências repetidas, e ainda carrega o overhead da tabela de frequências no cabeçalho.

2. Taxa de compressão — LZW

- **Tamanho original:** 13.856 bytes
- **Tamanho comprimido:** 5.222 bytes
- **Cálculos:**

1. Taxa (comprimido/original) = $5222 / 13856 = 0,3769 = 37,69 \%$
2. Economia de espaço = $1 - 0,3769 = 62,31 \%$
3. Razão de compressão = $13856 / 5222 = 2,65 : 1$

O que isso significa? O LZW reduziu o arquivo em aproximadamente 62%, realizando uma compressão melhor que o Huffman, neste caso. Os arquivos de dados têm muitos padrões repetidos (sequências longas de zeros no padding, cabeçalhos e campos similares entre registros). O LZW é baseado em dicionário e captura essas sequências repetidas inteiras com um único código de 16 bits, algo que o Huffman (byte a byte) não consegue realizar.

3. Comparação Direta e Conclusão

Algoritmo	Tamanho Original	Tamanho Comprimido	Redução do Arquivo
Huffman	13.856 bytes	7.605 bytes	Reduziu ~45%
LZW	13.856 bytes	5.222 bytes	Reduziu ~62%

Conclusão: Para este tipo de dados (structs binários com muitos espaços repetidos), o LZW é superior ao Huffman (LZW > Huffman).

Observação importante: Se o arquivo fosse muito pequeno, o Huffman poderia fazer com que o arquivo final ficasse maior do que o original. Isso ocorre porque a tabela que ele precisa salvar junto com o arquivo ocuparia mais espaço do que a compressão conseguiria salvar. Esse problema é solucionado em situações em que o volume cresce, como ocorreu neste teste.

Dificuldades enfrentadas ao implementar Huffman e LZW:

Huffman: Os I/O em nível de bit englobam arquivos que lidam com bytes; no entanto, os códigos apresentam tamanho variável em bits. Esse problema foi resolvido com um acumulador (unsigned char) além de um contador de bits, gravando o byte a cada 8 bits e aplicando padding no último byte.

LZW: Em algumas situações, o decodificador recebe um código que ainda não existe no seu dicionário (o codificador está “um passo à frente”). Esse problema foi solucionado com o tratamento clássico $\text{entry} = \text{previous} + \text{previous}[0]$.